

## RELEASE NOTES

# dockwatch

Self-hosted container update monitor for local Docker and Portainer-managed hosts. Entries are grouped by release, then by date, so it's easy to see what shipped in a given week.

|                                                                                      |                                                                                       |                                                                                     |
|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <div><div></div><div></div></div> <div>CURRENT VERSION</div> <div><b>0.7.0</b></div> | <div><div></div><div></div></div> <div>PREVIOUS VERSION</div> <div><b>0.6.0</b></div> | <div><div></div><div></div></div> <div>GENERATED</div> <div><b>2026-08-12</b></div> |
|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|



---

# 0.7.0

Released 2026-08-12

**August 12, 2026**

THIS WEEK

## ■ FIXED

- **Portainer mutation routes now respect `portainer.enabled`** — restarting or deleting a Portainer-managed container, and Portainer stack updates, previously ignored the integration's enabled/disabled toggle entirely; only container *discovery* checked it. Disabling Portainer in Settings now actually blocks these actions instead of silently allowing them.
- **Stack deploy calls now use a dedicated, longer timeout** — `create_stack` / `update_stack` block on Portainer's synchronous image pull + recreate, which routinely exceeds the short per-call timeout for real (non-trivial) images. Live-verified against a real Portainer instance: the deploy was completing successfully server-side even after the client had already raised a timeout error, producing false failures. Added `portainer.deploy_timeout` (default 120s, configurable) used only for these two calls.
- **Security hardening** — rate-limiting on mutating routes (container update/delete/restart/rollback, settings PUT, Portainer test, user creation) at 10 calls/60s per IP+path; structured logging of auth events (login success/failure, logout, registration, rejected sessions, permission-denied); daily SQLite backups via the WAL-safe online backup API, retaining the last 7 copies.

## ■ DOCS

- Documented the `portainer.deploy_timeout` config and the retry caveat (a timed-out redeploy call often still completes server-side — check Portainer's stack state before retrying).

**August 8, 2026**

## ■ ADDED

- **Opt-in per-container auto-update** — a new `auto_update` container flag (SQLite, alongside pinned/ignored), toggled per container from Settings → Monitoring Scope. On each scheduled check, flagged containers that come back outdated are updated automatically through the same plan/execute path as a manual click — same safety checks (pinned/floating-tag/compose guards), same audit log entry, attributed to `scheduler (auto-update)`. Off by default; nothing changes for containers not opted in.

**August 5, 2026**

■ **ADDED**

- **Portainer stack creation API** — `PortainerClient.create_stack()` deploys compose stacks programmatically via the Portainer API ( `POST /api/stacks/create/standalone/string` ), complementing the existing find/read/update stack methods. Includes a runnable `deploy_to_portainer.py` example in the README that reads a compose file and adjacent `.env` , then deploys it as a Portainer-managed stack with automatic source tagging.
- **Background scheduled check in web server** — the `serve` command now runs a background asyncio task that periodically checks all containers on the configured schedule, keeps the results cache warm, and broadcasts fresh data to connected dashboards via WebSocket. No need to click Refresh to see current state.

■ **CHANGED**

- **Source filter is now client-side only** — switching between Local / Portainer / All filters no longer triggers a full `check_all()` API call. The dashboard filters already-loaded results in memory, eliminating unnecessary Docker Hub registry calls.

■ **FIXED**

- **Multi-arch digest comparison** — `RepoDigests` always records the manifest-*index* digest for a multi-arch pull, but platform-selection logic was comparing that against a platform-specific manifest entry digest — two digest tiers that essentially never match, permanently misreporting multi-arch images (e.g. linuxserver.io images) as outdated even when content was byte-identical.
- **Compose workdir double-prefixing** — pasting an already-resolved host path (e.g. copied from an error message showing `/hostroot/ ...` ) into Settings caused `resolve_host_path()` to prepend the prefix again on every subsequent use, producing `/hostroot/hostroot/ ...` and failing the directory-exists check. Paths are now normalized on save.
- **Dashboard stat cards ignored the "All" filter** — Total / Up-to-date / Outdated / Pinned counts only reflected the currently selected source (Local or Portainer) instead of all containers; now always shows the true total regardless of toolbar filter.
- **Manifest digest & token caching** — Docker Hub manifest digests and `auth.docker.io` bearer tokens are now cached in-memory for 60 seconds, dramatically reducing API calls and eliminating 429 rate-limit errors on rapid successive checks (page refreshes, source/environment switches, auto-refresh).
- **409 race condition on refresh** — concurrent container checks are now properly guarded: `Toolbar` sets `isChecking` optimistically before the WebSocket round-trip, and the redundant `initialCheck` mutation is removed from `DashboardPage` . The auto-refresh interval also skips firing when a check is already in flight.

- **Portainer identity tracking** — containers are now tagged with the correct deployment source based on compose labels ( `/data/compose/` prefix for Portainer, local workdir paths for direct Docker). Portainer-discovered containers with local compose labels are no longer mis-tagged as "portainer". Source identity survives across check cycles and is properly deduplicated when the same container is visible on both sources.
- 

## 0.6.0

Released 2026-08-05

### ■ ADDED

- **Delete container & image** — admins (with a new `delete_containers` permission) can delete a container directly from the dashboard, for both local Docker and Portainer-managed containers, plus delete the underlying image for local containers. Both actions require confirmation and are logged to the update history like every other action.
  - **Update history & rollback** — every update/rollback attempt is recorded (who, when, old→new tag, success/failure) in a new `update_history` table, visible via a per-container history panel (admin-gated) with a one-click rollback for compose-managed containers.
  - **Digest drift alerts** — when a floating tag (e.g. `latest`) silently points at a new image digest, it's now surfaced as a distinct `digest_drift` notification and history entry instead of being folded into an ordinary update event.
  - **Portainer container restart** — Portainer-managed containers can now be restarted directly from the dashboard ( `POST /containers/{name}/restart` , proxied through Portainer's Docker API).
  - **Multi-arch-aware digest comparison** — outdated/drift detection for multi-arch images (manifest lists) now compares the digest of the platform actually deployed (matched via the local Docker daemon's reported architecture) instead of the manifest list's own digest, which changes whenever *any* platform is rebuilt. Falls back to the previous behavior when the daemon is unreachable or the platform isn't present in the list.
  - **Multi-user RBAC** — self-service registration, custom roles, per-route permission checks, user/role CRUD with last-admin protection.
  - **Authentication** — username/password login via PBKDF2-SHA256 cookies, login logout, and login/register pages in the dashboard.
  - **Container checkboxes** — ignored-containers text input replaced with a discovered-container checklist in Settings.
-

- **SQLite-backed pinned/ignored** — `container_flags` table replaces TOML fields for pin/ignore, with one-time migration from legacy config.
- 

- **Portainer compose stack updates** — Portainer-managed compose containers can now be fully updated (pull + recreate) via the dashboard. Previously only local Docker containers supported in-place updates; Portainer containers were limited to restart-only.

## ■ CHANGED

- **Settings page streamlined** — duplicate "Pinned containers" text field removed (dashboard row buttons are the canonical path); Portainer & Trivy sections collapsed under an "Advanced" disclosure.
- **Docker Hub tag lists cached** — Docker Hub REST API tag lists are now cached in-memory for 5 minutes per image, avoiding rate-limit errors on repeated checks. Tag list pagination shared across containers for the same image within a single scan.

## ■ FIXED

- Permission-gated UI controls now hidden from users lacking the permission (Nav, row actions, direct page access).
- Trivy severity bars are now clickable buttons that filter the findings list; zero-count severities disabled.
- Trivy vuln-DB and layer cache now persist across container recreates via a named volume.
- Malformed container records no longer crash the entire dashboard — error boundaries scope failures to individual rows.
- `PUT /settings` now returns 422 (not 500) on non-numeric field values.
- Numeric settings fields now coerce their values, preventing silent string storage.
- Stray debug `print()` in registry replaced with `logger.debug`.
- `_safe_version` now parses distro-suffixed tags ( `-alpine` , `-slim` , `-bookworm` ) via semver fallback.
- Pin/unpin config read-modify-write race fixed with SQLite serialization.
- Compose tag-rewrite regex no longer risks corrupting a sibling service's image line.
- Update button now rewrites the compose file's image tag, translates Windows host paths for Docker Desktop, and runs `docker compose pull / up -d` (or a full Docker SDK recreate for non-compose containers) so the container actually restarts on the new image.
- Stale wheel cache fix: `uv pip install` now forces reinstall of the dockwatch package.
- Pinned/ignored TOML migration now runs on CLI startup, not just on web-server start.

- `DOCKWATCH_USERNAME` / `DOCKWATCH_PASSWORD` now actually reach the container — `docker-compose.yml` never passed them through, so the documented "set these in `.env`" bootstrap path silently did nothing, leaving self-service registration open indefinitely on any install that followed the quickstart as written.
- `dockwatch serve` 's startup warning now checks the actual users table instead of the stale legacy single-credential config, and warns explicitly that an empty users table means the next visitor to register becomes admin.
- Upgraded `react-router-dom` to 7.18.2, fixing an open-redirect/XSS advisory (GHSA-jjmj-jmhj-qwj2) and an SSR-hydration constructor-injection advisory (GHSA-337j-9hxr-rhxg) present in the prior 6.x pin; upgraded `postcss` to fix a source-map path-traversal advisory (GHSA-r28c-9q8g-f849).
- Pinned all Dockerfile base images ( `python:3.12-slim` , `node:22-slim` , `docker:27-cli` , `aquasec/trivy` , `ghcr.io/astral-sh/uv` ) to a specific version and digest — two of these were previously floating on `:latest` , so a malicious or broken upstream push could silently change the next build.
- Portainer-discovered containers now resolve repo digests correctly for outdated/drift comparison.
- Update button hidden on Portainer-sourced rows for containers whose update path is not yet supported.
- New permissions are now synced into the existing admin role on startup, so upgrades automatically grant admins access to newly introduced features.
- Auth hardening: session cookie scoped consistently, registration endpoint returns correct error codes, Portainer settings endpoint requires authentication.